

Ship code to the **network**, not to a server.

Serverless functions that run on Cloudflare's global network, scale automatically from zero, and connect to data, storage and AI through bindings.

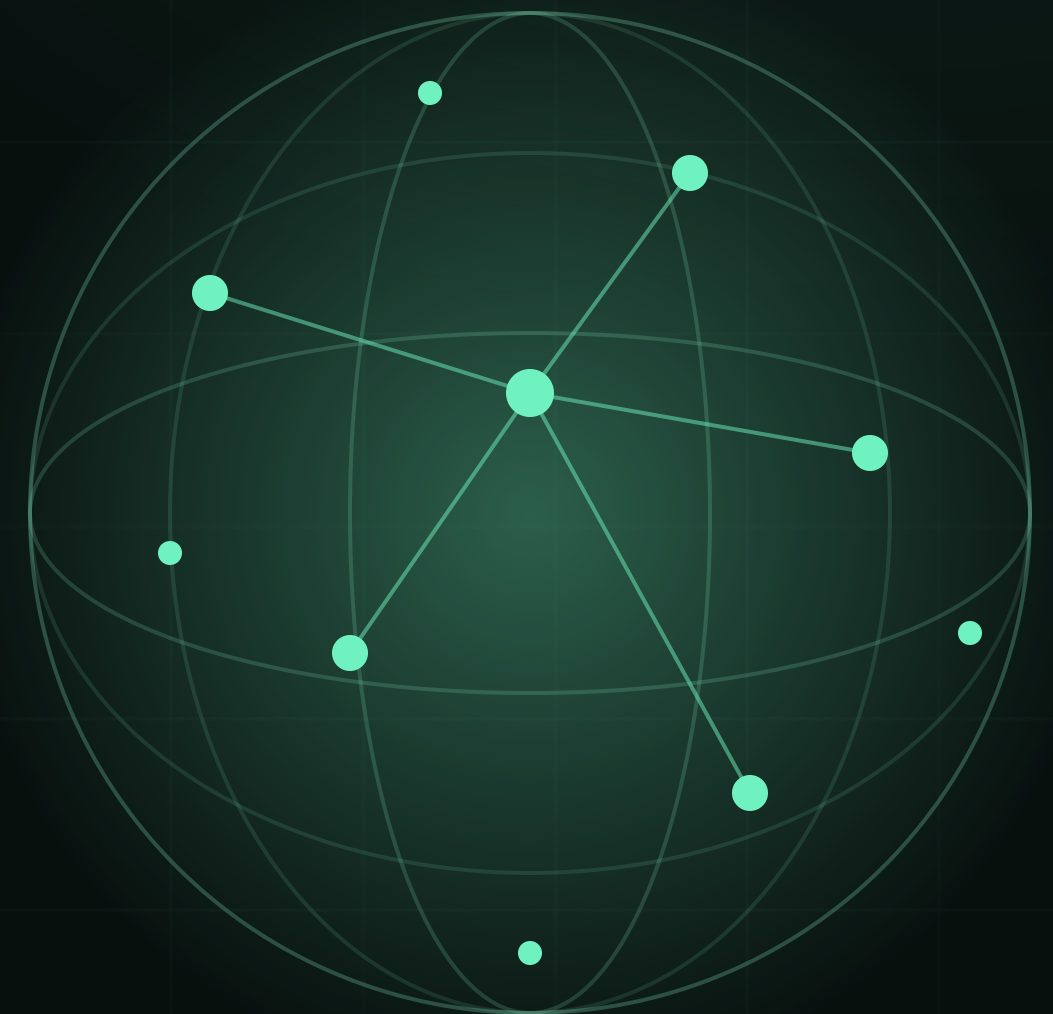
JavaScript

TypeScript

Python

Rust

Source: workers.cloudflare.com



From localhost to global.

The loop stays familiar: write, test locally, ship, and control the rollout. Deploy from the CLI or by merging in Git.

01

Start from a template

Pick a starter in JavaScript, TypeScript, Python or Rust.

02

Test it locally

Run the full app on your machine with `workerd`, the open-source runtime.

03

Deploy from CLI or Git

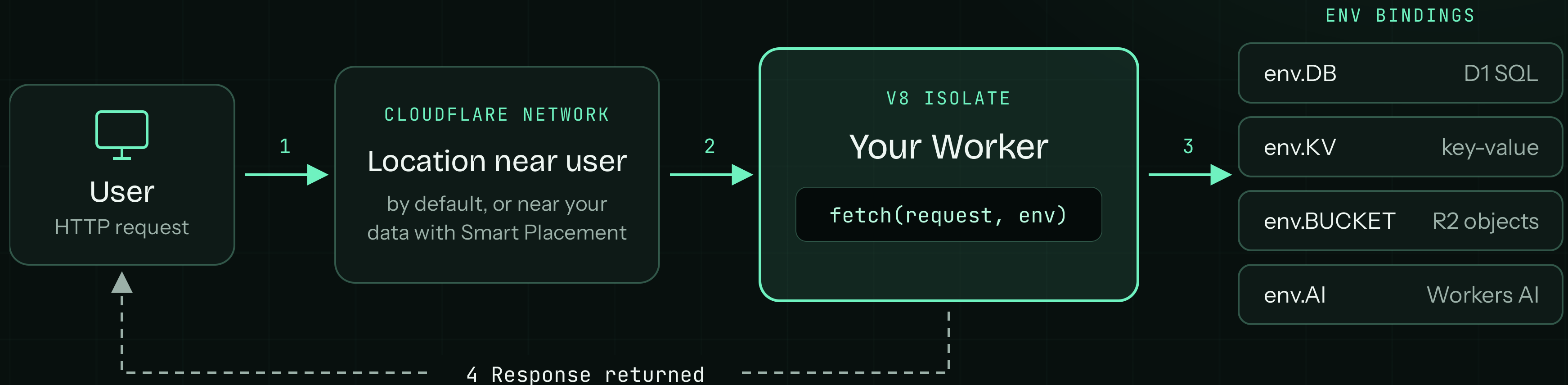
Push from the terminal, or connect a repository and deploy on merge.

04

Roll out, roll back

Ship everywhere at once or to a percentage of users; roll back if errors spike.

Deploy once. Run **everywhere**.



Deploy once

Code runs across Cloudflare's cities by default. No regions to configure.

Isolates, not containers

Lightweight V8 isolates scale up and down quickly, without process overhead.

Scale from zero

Concurrency follows demand; no pre-provisioning or prewarming.

Full stack, via bindings.

● ● ● index.ts

```
export default {
  async fetch(request, env) {
    const stmt = env.DB.prepare(
      'SELECT * FROM comments LIMIT 3');
    const { results } = await stmt.all();

    return new Response(renderHtml(results), {
      headers: { 'content-type': 'text/html' },
    });
  },
} satisfies ExportedHandler<Env>;
```

Query D1 from a Worker. The database arrives as a binding on env; no connection string in code. Adapted from the product page.

STORAGE & DATA

D1

KV

R2

Queues

Hyperdrive

STATE & ORCHESTRATION

Durable Objects

Workflows

Containers

AI

Workers AI

Vectorize

AI Gateway

Agents

Also: Browser Rendering, Images and more through the same binding model.

Where Workers **fit**.



Scalable APIs

Automatic scaling and global deployment, without load balancers or regional config.



Full-stack web apps

React, Vue or Next.js with backend logic, databases and storage alongside.



Edge logic

Auth, rate limiting, routing and caching near users, offloading your origin.



Background jobs

Webhooks, data processing and scheduled tasks with Queues, Workflows and Cron Triggers.

EVALUATE IT

Start with one route.

Move a single API endpoint or edge rule onto a Worker, test it locally, then roll it out to a share of traffic.

workers.cloudflare.com

developers.cloudflare.com/workers